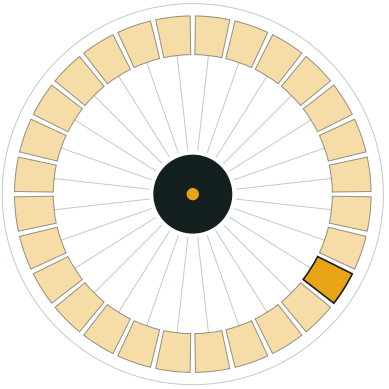
Managing Error in the Age of AI

From the Panopticon to a Learning Culture



Author Martín Dopazo

Version Draft v3 — October 28, 2026

Summary

With AI we write code much faster than we can review it, understand it and take ownership of it. The analysis that used to happen before production (possible paths, edge cases, business rules) moves into applications that are already in use. My argument in this paper is that management's answer cannot be more control, but an environment where error is seen, discussed and valued as a source of knowledge. I look at how the monitoring tools we use every day push people to hide errors, I propose telling blame apart from responsibility, and I leave three lines of action with two concrete tools: a blameless post-mortem template (Annex A) and guidelines for dailies without surveillance (Annex B).

1 The problem: code travels faster than understanding

Code is produced at a speed that doesn't leave room to evaluate it, analyze it and review it, and this happens at every stage of control:

- **In the developer's own debugging**, who accepts whatever "seems to work".
- **In tests**, which AIs tend to steer toward the happy path.
- **In Pull Requests**, which often aren't reviewed in depth: they get skimmed, and someone checks that it compiles.

The result is that the moment of analysis (possible paths, edge cases, business rules) moves to production. Problems show up with the system already in use, in front of the users.

Ariel Fabian (2026) named this phenomenon the **"bullet effect"**, by analogy with ultra-fast chess, where the lack of time rules out deep calculation. His thesis is that the problem isn't the tool but the pace: a person needs time to understand how software works, and can't do it at the speed at which a model generates text. When that happens, **cognitive debt** piles up: systems that work but that nobody on the team knows how to change safely, and whose failures can go straight to production. Fabian backs the diagnosis with recent evidence that he cites:

- Across 196,000 Pull Requests, deep human review dropped from 48% to 21% (He and Vasilescu, 2026).
- In organizations with massive AI adoption and no rules, changes approved without prior review rose 76.3% and critical production incidents rose 125.4% (Faros AI, 2026; this is an observational pattern).
- In a controlled trial with developers, those who delegated to AI understood 17 percentage points fewer of the concepts (Shen and Tamkin, 2025).

Fabian proposes five working rules adapted from chess (pace control, the 10 + 10 rule, a "cognitive toll" before the commit, preventive audit and the centaur model). And he acknowledges a limitation that, for me, is the starting point of this paper: *no technical rule works in isolation if the tracking indicators only look at immediate delivery volume*; when the pressure of the calendar beats the space for analysis, any control becomes a formal step that gets completed just to tick the box.

I start from that diagnosis and focus on **the other half of the problem**: what happens to a team when, on top of speed pressure, it works in an environment where making a mistake has consequences. The rules about pace and absorption (the cognitive toll, the 10 + 10) only work if the team can say "I don't understand this" without paying a price, and if what gets measured doesn't reward apparent speed. No technical rule guarantees that: it's a management decision.

2 Hypothesis

> Teams that make error visible, analyzable and valued understand better what they deliver, and that understanding is the best defense we have against the cognitive debt of AI-generated code.

From this comes a challenge for those of us who manage teams: to create an environment of **review, rethinking, cooperation and transparency** (*disclosure*) about what we know and what we don't know about the model, the architecture and the features we are building.

And, at the same time, to avoid the cognitive debt of using code that isn't understood or wasn't reviewed. Its consequence is the **absence of architectural purpose**: teams that build and support systems made at full speed, that nobody understands and that nobody owns.

3 Why teams hide error

3.1 Management styles

There are several ways to lead a development team. Some look for whoever is responsible for the error. Others downplay it, but still measure each developer's "productivity" by the errors they make.

And there is a third attitude that seems healthy but also deserves some distrust: "*it doesn't matter who made the mistake, I'm not looking for people to blame but for solutions*". Without meaning to, that stance **veils the error in favor of the solution**: it gets fixed, but nothing is learned about the development or about why the error happened. The problem is solved and the learning is lost.

3.2 The panopticon as a management metaphor

Excessive control (permanent meetings where people must account for progress and delays) achieves the opposite of what it seeks: problems get *hidden*, or people write code that "saves" what is visible without anyone knowing precisely what it does or what other problems it may bring.

Michel Foucault, in *Discipline and Punish*, describes the panopticon as a device applied to schools, hospitals, factories and barracks to order, correct and normalize behavior. What makes it effective is that control is **internalized**: since we never know for sure whether we are being watched at this moment, we end up acting as if we were always being watched.

Byung-Chul Han (*Psychopolitics*, *The Burnout Society*) adds that power today works less through external discipline than through voluntary exposure and self-exploitation: the digital panopticon is subtler and more effective because it needs no watcher. That difference matters here. In a development team the pressure almost never comes from a boss who watches, but from a board that updates itself and an availability that is put on display.

> **A note on the metaphor.** I'm not saying that Jira, dailies or pair programming are surveillance in themselves. I'm saying that, **when they are used to control the person and not to coordinate the work**, they produce the effects I describe here. I come back to this in section 8.

3.3 Tools as part of the device

Tool or practice	Stated purpose	Possible real effect
Background apps (Teams, Discord)	Make immediate communication easier and unblock people	Monitoring of idle time
Permanent voice channels	Simulate the closeness of the physical office when remote	Loss of privacy and of space to concentrate
Repository metrics (commits, lines, PRs)	Measure the stability and progress of the project	Quantifying human effort as if it were performance

What this looks like in practice:

- **Constant presence.** Statuses ("Available", "Away", "In a meeting") work like the lit windows of Bentham's cells. We know our availability is visible in real time, and that creates the pressure to be "always on".

- **Meetings and permanent voice.** Dailies that run long, mandatory pair programming and voice channels left open all day are justified as "organic collaboration", but they expose the time and thinking pace of whoever is programming and, with that, their possible mistakes.
- **The panopticon in the code.** The commit history (frequency, lines, delivery time) and the management metrics (time per task, burndown) end up working as performance records. The number of commits or PRs matters more than the architectural decision of organizing them so they can be understood and reviewed.

The AI aggravating factor. AI-assisted generation inflates exactly the activity metrics: more commits, more lines, more PRs in less time. If we use those metrics to evaluate, we measure more and more noise and less and less understanding. And metrics of AI *usage* (acceptance of suggestions, volume generated) are becoming a new control board.

This is already happening. According to specialized press, Meta came to include AI token consumption in its engineers' evaluations: there were leaderboards, and people took on extra tasks just to improve their score, which became known as *tokenmaxxing*. In September 2026, according to an internal memo, the company decided to stop using AI adoption dashboards and token counts to evaluate impact, and to go back to looking at the quality of the result, speed and the complexity of the problems (Cooter, 2026). Its own CTO had already warned that token usage, on its own, doesn't measure impact of any kind. It is Goodhart's law live: when a measure becomes a target, it stops being a good measure. That same law was cited by a fintech manager in a survey about AI usage dashboards, which also recorded developers' distrust of such detailed data on how they use their tools (Lazzaro, 2025).

3.4 The consequence: self-censorship

The deepest effect isn't the possible punishment but **self-censorship**. In software development it shows up in very concrete behaviors:

- **Productivity theater.** Visible tasks (moving cards, writing in public channels) get priority over deep work and complex problems, which need time away from the noise.
- **Availability anxiety.** The fear of showing up as "Away" fragments attention: the flow is cut to answer alerts and code quality is sacrificed to demonstrate presence.
- **The illusion of autonomy.** Agile methodologies are presented as self-organization frameworks, but if they depend on strict data monitoring and constant meetings, autonomy becomes **coordinated self-surveillance**: the team disciplines itself just to satisfy the board's metric.

Programming requires abstraction and, above all, understanding the problem. The digital panopticon asks for something else: visibility and communicative noise. The result is code that works today and that nobody can explain tomorrow, plus a burnout that is paid for by people.

4 Error as an object of knowledge

To build an environment that is friendly to error and to what we don't know, I rely on **constructivism**, the current in learning theory according to which knowledge isn't received ready-made but constructed, and error is part of that process. Emilia Ferreiro, a student of Piaget, showed this with how children learn to write: what looks from the outside like children's mistakes are hypotheses of their own, coherent ones, which they keep revising until they get closer to the convention (Ferreiro and Teberosky, 1979). An error isn't necessarily a flaw or a failure: it is, most likely, **evidence of a hypothesis in crisis**, the logic behind the error.

The underlying strategy is to **accept, value and discuss** error: not to justify or defend ourselves, but to explain it. And not only what was functional, but **the logic that led us to produce it**.

An error can have many origins:

- a badly worded prompt;
- a poor analysis of the business rule the software has to meet;
- edge cases that weren't considered;
- environments with exceptional conditions (no connectivity, database down, etc.);
- an infrastructure that can't support the requirements.

Only if we value and accept error do we come to understand the model and the flows of the software in action. And that, in addition, helps us anticipate problems more precisely and diagnose better the ones that appear.

5 Responsibility without blame

A culture without blame faces a predictable objection: if nobody is to blame, who takes charge? My answer is that **blame and responsibility are different things**, and that a culture of error needs the second precisely in order to do without the first.

	Blame	Responsibility
Looks toward	The past: the damage already done	The future: repair and prevention
Asks	Who did it? What do they deserve?	What happened? Who needs what? What do we change?
Effect on the team	Information gets hidden	Information gets shared
Result	Punishment	Learning and improvement of the system

The distinction has precedents. Sidney Dekker, in his work on *just culture* (*Just Culture: Balancing Safety and Accountability*), contrasts a **retributive** version, which classifies behaviors and asks what punishment is due, with a **restorative** version, which asks who was affected, what they need and whose job it is to attend to it. John Allspaw, when he introduced blameless post-mortems at Etsy (2012), framed it in management terms: look at errors "from a learning perspective" and balance safety and accountability, not eliminate the latter.

Applied to development with AI, this translates into a concrete rule:

> **Whoever approves takes responsibility.**
>
> They must be able to explain what the change does, why it does it and which cases it doesn't cover, even if they didn't write it. "It was the AI's mistake" is not an explanation, neither for whoever develops nor for whoever manages.

The rule doesn't punish the error: it **points at opacity**. If someone can't explain a change, what should be expected is not a sanction but a repair: read it together, write the missing test or undo the change. What is protected is the team's understanding, not anyone's reputation.

This doesn't mean I'm unaware that there are behaviors no healthy culture condones, such as deliberate negligence; I leave it outside this paper. What I do propose is a precaution before pointing at someone: ask whether the system rewards that behavior.

> Whoever approves a PR without reading it, in an environment where closed PRs per week are measured, isn't breaking the rules: they're following them.

Finally, there is one irresponsibility that this culture does want to eliminate: **hiding the error**. Whoever hides it takes charge neither of the failure nor of what the team could learn from it. That is why psychological safety is not the opposite of responsibility: it is what makes it possible.

6 Proposal: three lines of action

Team management has to move from a culture of **presence monitoring** to a culture of **impact and psychological safety**. In software, error isn't a personal flaw: it is an unavoidable by-product of exploring, of trying to solve problems and of innovating. With these strategies, management stops being the central watchtower and becomes a **facilitator of infrastructure and trust**, so that building software can once again be a creative and iterative discipline.

6.1 Reconfigure communication and presence

- **Sovereignty over availability status.** An explicit "no indicators" policy: it should be normal to close the messaging app or switch to "Do Not Disturb" for blocks of 3 or 4 hours of deep work.

- **Asynchronous communication by default.** A reply can take 1 or 2 hours without that meaning a lack of commitment. For urgent matters, a specific escalation channel is defined (a phone call, for example).
- **No permanent voice rooms.** Replace them with limited "office hours" (30 minutes at the start or end of the day, for example) or with one-off design rooms. Collaborating has to be a conscious decision, not a continuous exposure.

6.2 Redefine metrics

- **Impact, not activity.** Avoid lines of code, hours online or number of commits as a measure of performance. Measure the value delivered to the user, the stability of the system, the fulfillment of sprint goals and, above all, **the technical understanding of what is contributed to the system**.
- **The board as a map, not a whip.** Management tools are there so the team can coordinate dependencies, not to micromanage every task. If a card falls behind, the question is "*how can I help you unblock it?*" and not "*why did it take you longer?*". And if the blockage reveals something relevant about the system, it is analyzed in depth and, once resolved, the finding is shared with the team.

6.3 Institutionalize error as a natural part of the work

For the team to stop hiding failures out of fear of surveillance, error has to be visible, analyzed and understood as a source of improvement.

- **Blameless post-mortems.** After a serious production failure or a critical bug, the meeting concentrates on how and why the system failed, not on who made the mistake. It looks for process failures, missing automated tests or ambiguity in the requirements. In teams that use AI, the analysis includes the **prompt, the context it was given and what was actually reviewed** (template in Annex A).
- **Safe experimentation environments.** Solid local or staging environments where breaking the code is cheap and safe. If I know that an error in my branch has no punitive consequences or negative exposure, I dare to look for better architecture solutions.
- **Shared learning sessions (FailFaire).** Monthly spaces where tech leads and senior developers share their recent mistakes and what they learned. Seeing the people we look up to be vulnerable dismantles the panoptic illusion of mandatory perfection.
- **Dailies without surveillance.** The daily stops being an individual report and becomes an inspection of the team's work (guidelines in Annex B).

7 How to know whether it works

A proposal about metrics has to be measurable, taking care not to reproduce the problem it criticizes. Some possible indicators:

- **Delivery stability:** change failure rate and recovery time (DORA metrics).

- **Learning:** number of post-mortems held and of preventive actions actually closed.
- **Understanding:** percentage of changes where whoever approved was able to explain them, sampled and not person by person.
- **Climate:** a short, anonymous survey on psychological safety (for example, Edmondson's scale).

Measuring the adoption of a blameless culture with individual, permanent instruments would be creating a new panopticon. That is why the indicators are collected **at team level, by sampling and with a stated purpose**.

8 Limits and objections

A management proposal has to be open to discussion. These are the strongest objections I see and how I answer them, or leave them open.

8.1 Not all control is surveillance

A team needs to coordinate, and remote teams even more so. Asynchronous communication avoids interruptions, but it has costs: delays, misunderstandings, decisions waiting on a reply. And there are moments (an incident in progress, a design under discussion) that call for a real-time conversation. That is why I don't propose eliminating synchronicity but **making it deliberate**: meetings with a purpose and limited duration, and clear escalation channels for urgent matters. At the same time, keep in mind the following statement: **If everything is urgent, nothing is urgent**.

8.2 The tools are not the problem

Neither Jira, nor dailies, nor pair programming are surveillance in themselves. The same board can be a map that helps coordinate or a whip that measures minutes. The panopticon metaphor describes a **use** of the tools (controlling the person), not the tools.

8.3 A record for auditing is not the same as a record for evaluating

Many environments, especially regulated ones, **require traceability**: who changed what and when. I'm not proposing to give up that record, but to separate its purpose: the history serves to audit and to reconstruct an incident, not to measure people's performance or to build rankings of errors.

8.4 Traceability as responsibility, not as blame

Tracing which change caused an incident and who approved it is a legitimate and necessary practice. Experimental designs on development with AI, like the protocol Fabian proposes (2026), take it into account by tracking the commit that caused a failure and recording the authorship of the code. From my perspective, that audit is understood as **responsibility** (the ability to reconstruct what happened, understand why and repair it) and not as **blame**. The difference isn't in the data but in the use: the same history can serve to learn or to point fingers. For it to remain the former, indicators are aggregated at team level, reported before starting and not used to evaluate people.

8.5 A team doesn't change alone

A team can adopt dailies without surveillance and blameless post-mortems, but if the organization around it evaluates by closed tickets, generated lines or delivery speed, the blameless culture ends up as a mere veneer. Fabian (2026) warns of something equivalent about his rules: no control works in isolation if the indicators only reward immediate volume. What is needed is **coherence across levels**: what the team says, what management measures and what the organization rewards.

8.6 The risk of the empty ritual

A blameless post-mortem that produces no closed actions is just another form of theater. The same goes for a "no surveillance" daily that is still an individual report under another name. That is why the annexes ask for actions with an owner and a date, and a periodic review of whether they were carried out.

8.7 Scope and evidence

This proposal rests on theory (learning, just culture, psychological safety) and on my experience managing development teams. There is also no single measure of "error culture", and it is likely that context (team size, sector, level of regulation, maturity) changes its effect.

As a future line, Fabian's factorial design (reserved absorption time and cognitive toll) could add a third factor: the team's climate toward error. The question would be whether the pace and understanding rules work better, the same or worse in teams with high psychological safety. It is a hypothesis, not a result.

9 Conclusion

AI didn't invent the problem of error in software teams, but it made it more costly. When code is produced faster than we can understand it, what wasn't reviewed reaches production and what nobody understands has no owner. The pace and absorption rules, like Fabian's, attack that speed. I argue that those rules need a precondition: **an environment where saying "I don't understand this" costs nothing.**

That environment isn't achieved with more control. Monitoring tools, permanent meetings and activity metrics, when used to watch the person and not to coordinate the work, push people to hide problems and to produce code that works without anyone knowing why. The alternative I propose rests on three ideas:

1. **Error is evidence of a hypothesis in crisis**: an object of analysis and learning, not a reason for shame.
2. **Responsibility is not blame.** Whoever approves takes responsibility for understanding what they deliver; the analysis of a failure concentrates on how the system allowed it.
3. **What is measured and rewarded has to be coherent with what is said.** Otherwise, the blameless culture stays as a declaration with no effect.

The concrete tools (blameless post-mortems, dailies centered on the board, impact metrics, spaces to share errors) are a starting point to try in real teams and adjust with data. I don't have evidence of my own yet, and saying so is part of being consistent: a model that asks us to value error has to be able to be wrong in public.

The speed of AI isn't going to slow down. What does depend on those of us who manage teams is deciding what happens when someone makes a mistake: whether it gets hidden or studied. The team's continued understanding of what it builds depends on that decision.

References

Fabian, A. (2026). *El Efecto Bullet: Deuda Cognitiva y la Ilusión de la Velocidad en la Era de la IA* (The Bullet Effect: Cognitive Debt and the Illusion of Speed in the Age of AI). Zenodo. DOI 10.5281/zenodo.23641059. https://arfab.github.io/efectobullet/ (In Spanish.)

Cooter, M. (2026, September 4). Meta minimizes role of token maxing in employee evaluations. *InfoWorld / CIO*. https://www.cio.com/article/4218661/meta-minimizes-role-of-token-maxing-in-employee-evaluations-2.html (Cites a report by The Information.)

Lazzaro, S. (2025, June 18). AI coding trackers are here. Proceed with caution. *LeadDev*. https://leaddev.com/reporting/ai-coding-tool-trackers-proceed-with-caution

Foucault, M. *Discipline and Punish: The Birth of the Prison*.

Foucault, M. *Microfísica del poder* (Power/Knowledge: Selected Interviews and Other Writings; a Spanish-language compilation).

Han, B.-C. *Psychopolitics: Neoliberalism and New Technologies of Power*.

Han, B.-C. *The Burnout Society*.

Dekker, S. *Just Culture: Balancing Safety and Accountability*. (Ashgate Publishing, 2007)

Allspaw, J. (2012). Blameless PostMortems and a Just Culture. Etsy, Code as Craft. (https://www.etsy.com/pl/codeascraft/blameless-postmortems)

Ferreiro, E. and Teberosky, A. (1979). *Los sistemas de escritura en el desarrollo del niño*. Siglo XXI. (English edition: *Literacy Before Schooling*, 1982.)

Edmondson, A. (1999). Psychological safety and learning behavior in work teams. *Administrative Science Quarterly*.

Kerth, N. (2001). *Project Retrospectives: A Handbook for Team Reviews*. (The Prime Directive.)

DORA / *Accelerate* (Forsgren, Humble and Kim).

Schwaber, K. and Sutherland, J. *The Scrum Guide* (2020).

Annexes

- Annex A. Blameless post-mortem template (Spanish)

- Annex B. Guidelines for dailies without surveillance (Spanish)